

Interview Preparation Guide

Deepesh Navani — Full Stack Developer

#	Section
1	Project — RAG: YouTube Playlist Knowledge Base
2	Project — AI Phishing Website Detection (ML)
3	Project — HireSense (AI Career Assistant)
4	Project — StrideX (E-Commerce Platform)
5	Experience — Odyssey Homes
6	Experience — AIBI (AI Multiverse, Loan App, Coupon Stall)
7	Experience — Sinjan Solutions (QR Attendance)
8	Experience — Insys Technologies
9	Resume-Based Questions

PROJECT 1 — RAG: YouTube Playlist Knowledge Base

Tech Stack: React.js, Tailwind CSS, Node.js, Express.js, Python, Chroma DB, LLM, Embeddings, MongoDB

1. One-Line Summary

An AI-powered video revision assistant — student submits a YouTube playlist, asks revision questions in natural language, and gets grounded answers from the actual lecture content with a direct timestamp link to the exact video moment.

2. 30-60 Second Interview Explanation

■ *Say this when asked 'Explain your project'*

The problem I wanted to solve was simple — students waste a lot of time manually scrubbing through long lecture playlists during revision. My project solves this using RAG — Retrieval Augmented Generation.

The user submits a YouTube playlist URL. The system uses yt-dlp to download the audio, Whisper to transcribe it to timestamped text, splits it into chunks, converts each chunk into vector embeddings, and stores them in Chroma DB — a vector database. Each chunk also remembers which video it came from and at what timestamp.

When a student asks a revision question, the system does a semantic search in Chroma DB — finds the most relevant chunks — and passes them to an LLM to generate a focused answer. Along with the answer, it gives a direct YouTube link with the exact timestamp so the student can jump straight to that part of the lecture.

Frontend is React.js, backend is Node.js, AI processing is in Python, and vector store is Chroma DB.

3. Detailed Feature Explanation

- **Playlist Ingestion:** User submits a YouTube playlist URL. System extracts all video IDs automatically. yt-dlp downloads the audio for each video — no need to download the full video, just the audio since we only care about what the teacher says.
- **Transcription:** Whisper (OpenAI's open-source speech-to-text model) converts each audio file to text with timestamps — every chunk knows its start time in seconds.
- **Chunking:** Full transcript split into smaller meaningful chunks (~70-75 seconds or 300-500 words) with overlap. Overlap ensures concepts that span chunk boundaries are not lost.
- **Embeddings:** Each chunk converted to a vector embedding — a list of numbers representing the semantic meaning of that text. Similar meanings produce similar vectors.
- **Chroma DB Storage:** Embeddings stored in Chroma DB along with metadata — video ID, timestamp, chunk text, video title. This becomes the searchable knowledge base.
- **Semantic Search:** User's question is also converted to an embedding. Chroma DB finds chunks whose embeddings are closest in meaning — not keyword matching, meaning matching.
- **RAG Pipeline:** Retrieved chunks passed as context to the LLM with the question. LLM generates a grounded answer using only that lecture content — not its general knowledge.
- **Timestamp Return:** Answer returned along with YouTube link at exact timestamp:
youtube.com/watch?v=VIDEO_ID&t=SECONDS. Student clicks and video starts at that exact moment.

4. Problem Statement

Students studying from long YouTube playlists struggle during revision — they remember a concept was explained somewhere but have to manually scrub through hours of video to find it. Also, if they directly ask an LLM, it answers from general knowledge which may differ from what their lecturer taught. This project solves both problems — find the exact lecture moment and answer from that specific content.

5. Complete Workflow

Ingestion Pipeline (runs when playlist is added):

User submits YouTube playlist URL → Backend extracts all video IDs

↓

yt-dlp downloads audio for each video

↓

Whisper transcribes audio → Timestamped text (start time per chunk)

↓

Text split into chunks with overlap

↓

Each chunk converted to vector embedding

↓

Embeddings + metadata (video ID, timestamp, title) stored in Chroma DB

↓

Playlist is now a searchable knowledge base

Query Pipeline (runs when student asks a question):

Student types question on React.js frontend

↓

POST /api/ask → Node.js backend → Python AI service

↓

Question converted to embedding

↓

Chroma DB semantic similarity search → Top relevant chunks returned

↓

Chunks + question passed to LLM as RAG context

↓

LLM generates grounded answer using only that context

↓

Answer + video ID + timestamp returned to frontend

↓

Student reads answer → Clicks timestamp link → Video opens at exact moment

6. Architecture

- **Frontend — React.js + Tailwind CSS:** Playlist submission, question input, answer display with timestamp link and video player.
- **Backend — Node.js + Express.js:** REST API layer. Coordinates between frontend, Python AI service, and MongoDB.
- **AI Processing — Python:** Separate service handling yt-dlp download, Whisper transcription, chunking, embedding generation, Chroma DB operations, LLM calls. Python used because it has best AI/ML library ecosystem.
- **Chroma DB:** Vector database storing embeddings + metadata. Handles semantic similarity search.

- **MongoDB:** Stores user accounts, playlist history, processed video list, saved questions/answers.

7. Tech Stack — Full Explanation

RAG (Retrieval Augmented Generation)

- **What:** A technique where instead of letting the LLM answer from its own training knowledge, you first retrieve relevant documents from your own data and give them to the LLM as context. The LLM then answers based only on that context.
- **Why:** Without RAG, the LLM answers from general knowledge — may be wrong or different from what your lecturer taught. With RAG, answers are grounded in the actual lecture content. This is the core differentiator of the project.
- **Where:** The entire question-answering pipeline.

yt-dlp

- **What:** A command-line tool for downloading audio/video from YouTube and other platforms. We use it to extract just the audio from lecture videos.
- **Why:** We only need what the teacher says — not the video frames. Audio is much smaller to process. yt-dlp is open-source, actively maintained, and handles YouTube's various formats reliably.
- **Where:** First step of ingestion pipeline — downloading audio from each playlist video.

Whisper

- **What:** OpenAI's open-source speech-to-text model. Takes audio input and returns accurate text with timestamps.
- **Why:** We need text to chunk and embed — Whisper does this accurately and is free to run locally. The timestamps it provides are essential for the timestamp-return feature.
- **Where:** Converting downloaded audio to timestamped text transcripts.

Chroma DB

- **What:** An open-source vector database — stores vector embeddings and allows semantic similarity search across them.
- **Why:** When a student asks a question, we need to find the most relevant lecture chunks by meaning — not keywords. Chroma DB is purpose-built for this. It's lightweight, easy to set up locally, and perfect for this use case.
- **Where:** Storing all chunk embeddings with metadata. Performing semantic search at query time.

Vector Embeddings

- **What:** A way to represent text as a list of numbers (a vector) that captures the semantic meaning. Similar meanings produce similar vectors that are mathematically close to each other.
- **Why:** Keyword search fails when student uses different words than the lecturer. Embeddings capture meaning — 'how does engine cool down' can match a chunk about 'thermal regulation' because they mean the same thing.
- **Where:** Every text chunk embedded before storage. Every user query embedded before search.

Python

- **What:** Programming language used for all AI processing tasks.
- **Why:** Python has the best ecosystem for AI — Whisper, sentence-transformers, Chroma DB, LangChain all have Python SDKs. Node.js doesn't have mature equivalents for these AI tasks.
- **Where:** All AI processing — transcription, chunking, embedding, Chroma DB operations, LLM calls.

Node.js + Express.js

- **What:** Backend server and REST API framework.

- **Why:** Handles API communication between React frontend and Python AI service. Node is fast for I/O and the team was already comfortable with it.
- **Where:** API endpoints — playlist submission, question handling, user management.

MongoDB

- **What:** NoSQL document database for application data.
- **Why:** Stores user accounts, playlist processing history, saved answers. Chroma DB handles vector data. MongoDB handles everything else.
- **Where:** User data, playlist metadata, answer history.

8. Key Concepts Explained

RAG — What it actually means

- R = Retrieval: Find the most relevant lecture chunks for the user's question from Chroma DB.
- A = Augmented: Add those chunks as extra context to the LLM prompt.
- G = Generation: LLM generates answer based on that specific context — not general knowledge.
- Without RAG: LLM answers from training data — can hallucinate or give generic answers.
- With RAG: LLM answers from your actual lecture — grounded, accurate, specific to your course.

Semantic Search vs Keyword Search

- Keyword search: looks for exact words. 'engine heat' won't find a chunk about 'thermal management'.
- Semantic search: looks for meaning similarity using embeddings. 'engine heat' WILL find 'thermal management' because they mean the same thing.
- This is why the student doesn't need to use the exact words the lecturer used.

Chunking and Overlap

- Why chunk? A 2-hour lecture is too large to embed as one piece — embeddings lose specificity.
- Why overlap? If a concept spans two chunks, overlap ensures it appears fully in at least one chunk. Without overlap, context at boundaries gets cut off.

How Timestamp Works

- Whisper returns text with timestamps — every segment has a start time in seconds.
- When chunking, we store the start timestamp of the first segment in each chunk.
- When that chunk is retrieved, we build: youtube.com/watch?v=VIDEO_ID&t=TIMESTAMP_SECONDS
- Student clicks → video starts at the exact moment that concept was explained.

Chroma DB vs Qdrant — Interview Answer

- Both are vector databases — store embeddings and do similarity search.
- Chroma DB: simpler, easier to set up, great for local development and smaller projects. Used in this project.
- Qdrant: more production-ready — better performance at scale, more filtering options, supports distributed deployment.
- Chroma DB was the right choice here. Qdrant would be the upgrade path for production scale.

Why Python separate from Node.js?

- Python has Whisper, sentence-transformers, Chroma DB SDK, LangChain — best AI libraries.
- Node.js handles the web API layer. Python handles all AI processing.
- Architecture: React → Node/Express (API layer) → Python service (AI processing) → Chroma DB.

9. Biggest Challenge

- **Challenge:** Long playlists take significant time to process — 20 videos × 1 hour each is a lot. User cannot wait on the same page.
- **Solution:** Made processing asynchronous. User submits playlist and gets confirmation. Processing runs in background. User is notified when knowledge base is ready.
- **Learned:** Long-running tasks should never block the main request. Use async processing with status tracking for heavy jobs.

10. Improvements

- Support multiple playlists per subject — student can ask across all at once.
- Add quiz generation — create MCQs from lecture content automatically.
- Support PDF lecture notes alongside video content.
- Switch to Qdrant for production — better performance at scale.

11. Follow-up Questions

Q: What is a vector database? Why not just use MongoDB?

A: A vector database stores and searches vector embeddings using similarity search. MongoDB does exact lookups — find document where ID equals X. Chroma DB does similarity lookups — find documents whose embeddings are closest to this query embedding. You cannot do semantic similarity search in MongoDB efficiently. Chroma DB is purpose-built for this.

Q: What is the difference between Chroma DB and Qdrant?

A: Both are vector databases. Chroma DB is simpler, easier to set up, and great for local development and smaller projects — which is why I used it here. Qdrant is more production-ready — better performance at scale, more filtering options, supports distributed deployment. Qdrant would be the upgrade path for production.

Q: What embedding model did you use?

A: I used sentence-transformers — specifically all-MiniLM-L6-v2 which converts text to 384-dimensional vectors. It's lightweight, fast, runs locally without any API cost, and gives good semantic similarity results.

Q: What is hallucination in LLMs?

A: When an LLM makes up information that sounds confident but is factually wrong. Without RAG, asking an LLM about a specific lecture topic gives a plausible-sounding answer from its training data that may differ from what your lecturer actually taught. RAG prevents this by forcing the LLM to answer only from the retrieved lecture chunks.

Q: How do you handle a question the playlist doesn't cover?

A: Chroma DB returns similarity scores with results. If the top result has a very low similarity score — no chunk is close to the question — we detect this and tell the user: 'This topic was not covered in the provided playlist' instead of generating a potentially wrong answer.

Q: What is LangChain?

A: LangChain is a Python framework that provides ready-made components for building RAG pipelines — document loading, text splitting, embedding, vector store integration, LLM chaining. Instead of writing all the plumbing manually, LangChain gives clean abstractions. I used it to connect the components of the pipeline.

Q: Why yt-dlp and not the YouTube API?

A: YouTube Data API gives metadata — title, description, comments. It does not give you the actual audio content. yt-dlp downloads the actual audio stream which is what we need for transcription.

Q: What is Whisper?

A: Whisper is OpenAI's open-source automatic speech recognition model. It takes audio input and returns accurate text with timestamps. It runs locally — no API cost — and handles accents and varied audio quality well.

Q: How would you scale this?

A: Switch Chroma DB to Qdrant for distributed vector search. Use a job queue like Celery with Redis for background processing of large playlists. Deploy Python service separately on a GPU instance for faster

Whisper transcription. Node.js backend scales horizontally behind a load balancer.

PROJECT 2 — AI Phishing Website Detection

Tech Stack: Python, Pandas, NumPy, SciPy, Scikit-learn, Matplotlib, Seaborn, Joblib

1. One-Line Summary

A supervised machine learning system that classifies websites as legitimate or phishing using 30 URL, domain, webpage, and reputation-based features from the UCI Phishing Websites dataset — using Logistic Regression inside a Scikit-learn Pipeline.

2. 30-60 Second Interview Explanation

■ Say this when asked 'Explain your project'

This project is an AI-based phishing website detection system. The idea is simple — a phishing website is a fake site that looks real and tries to steal user information. Instead of manually checking each site, I trained a machine learning model to automatically classify a website as legitimate or phishing based on 30 characteristics.

I used the UCI Phishing Websites dataset from Kaggle. The features include things like URL length, whether the site uses HTTPS, domain age, whether there's an IP address in the URL, form submission behavior, and web traffic — 30 features total.

I cleaned the data, engineered the binary target, did an 80-20 stratified split, and trained a Logistic Regression model inside a Scikit-learn Pipeline with StandardScaler. I evaluated using accuracy, precision, recall, F1-score, and confusion matrix. Finally saved the model as a .pkl file using Joblib for future inference.

3. Problem Statement

Phishing websites are a major cybersecurity threat. They look identical to legitimate websites but steal sensitive data like passwords and credit card numbers. Manual detection is slow and impossible at scale. An ML model can analyze website characteristics and classify it automatically in milliseconds.

- Example of phishing: www.amazon-login-security-verify.xyz looks similar to amazon.com but is malicious.
- The model takes 30 features of the website and outputs: 0 = Legitimate, 1 = Phishing.

4. Dataset — UCI Phishing Websites

- Source: UCI Machine Learning Repository — available on Kaggle.
- Format: ARFF file (Attribute-Relation File Format) — common in ML/data mining tools.
- Each row = one website with 30 feature columns + 1 target column.
- Original target: -1 = Phishing, 1 = Legitimate. Converted to: 0 = Legitimate, 1 = Phishing for binary classification.
- 30 features cover 4 categories: URL features, Domain/Security features, Webpage features, Reputation features.

5. The 30 Features — Categories

URL Features

- **having_IP_Address:** Is there an IP address in the URL instead of a domain name? Suspicious pattern.
- **URL_Length:** Very long unusual URLs are common in phishing.
- **Shortening_Service:** URL shorteners like bit.ly hide the actual destination — suspicious.
- **having_At_Symbol:** @ in URL like example.com@malicious.com is suspicious.
- **double_slash_redirecting:** Unusual // redirection patterns in URL.
- **Prefix_Suffix:** Suspicious prefix/suffix in domain name.
- **having_Sub_Domain:** Unusual subdomain structure patterns.

Domain and Security Features

- **SSLfinal_State**: SSL/HTTPS security state. Note: HTTPS alone doesn't guarantee legitimacy.
- **Domain_registration_length**: How long the domain is registered for. Phishing domains often have short registration.
- **age_of_domain**: How old the domain is. New domains are more suspicious.
- **DNSRecord**: Whether DNS record exists for the domain.
- **HTTPS_token**: HTTPS-related characteristics in the URL itself.
- **Favicon, port**: Favicon behavior and port-related information.

Webpage Behavior Features

- **Request_URL**: How many external resources the page loads.
- **URL_of_Anchor**: Behavior of links (anchor tags) on the page.
- **SFH (Server Form Handler)**: Where the login form sends data. If it sends to a suspicious URL — red flag.
- **Submitting_to_email**: Does the form submit data directly to an email address?
- **Redirect**: Number of redirects. Multiple unusual redirects are suspicious.
- **RightClick, on_mouseover, popUpWindow, Iframe**: Behavior patterns — disabling right-click, suspicious popups, hidden iframes.

Reputation Features

- **web_traffic**: Traffic level of the website.
- **Page_Rank**: Website ranking.
- **Google_Index**: Whether the site is indexed by Google.
- **Links_pointing_to_page**: How many external sites link to this page.
- **Statistical_report**: Statistical reputation-based information.

6. Complete Workflow

Load training_data.arff using SciPy → Convert to Pandas DataFrame

↓

Data Cleaning — handle missing values, remove duplicates

↓

Target Conversion — -1/1 → 0/1 (binary classification)

↓

Separate X (30 features) and y (target)

↓

80-20 Stratified Train-Test Split (stratify=y)

↓

Build Scikit-learn Pipeline: StandardScaler → Logistic Regression

↓

Train pipeline on training data

↓

Predict on test data

↓

Evaluate: Accuracy, Precision, Recall, F1-Score, Confusion Matrix

↓

Visualize: Confusion matrix heatmap, feature coefficient chart (Matplotlib/Seaborn)

↓

Save trained pipeline as .pkl using Joblib

7. Tech Stack Explained

Pandas

- **What:** Python library for data manipulation — loading CSVs/ARFF, filtering, creating columns, grouping.
- **Why:** Dataset is a file with thousands of rows. Pandas makes loading, cleaning, and transforming this data fast and easy.
- **Where:** Loading dataset, cleaning, target conversion, train-test split.

SciPy

- **What:** Scientific computing library for Python.
- **Why:** The dataset is in ARFF format — `scipy.io.arff` is the standard way to load this format into Python.
- **Where:** Loading the `training_data.arff` file.

NumPy

- **What:** Numerical computing library — array operations, math functions.
- **Why:** Pandas uses NumPy internally. Also used for numerical operations during feature engineering.
- **Where:** Numerical computations during preprocessing.

Scikit-learn

- **What:** Python's most popular ML library — algorithms, preprocessing, evaluation all in one.
- **Why:** Has everything needed — `train_test_split`, `StandardScaler`, `Pipeline`, `LogisticRegression`, all evaluation metrics. No need for multiple libraries.
- **Where:** Everything ML — split, scale, train, predict, evaluate.

Logistic Regression

- **What:** A classification algorithm that predicts probability of belonging to a class. Despite the name 'regression', it's used for classification.
- **Why:** Binary classification problem — legitimate or phishing. Logistic Regression is interpretable, fast to train, works well on structured tabular data with scaled features.
- **Where:** The main ML model.

StandardScaler

- **What:** Standardizes features to zero mean and unit variance.
- **Why:** Different features have very different scales — `URL_Length` might be 50 while `web_traffic` might be 100,000. Without scaling, large-scale features dominate. `StandardScaler` brings all features to the same scale.
- **Where:** Preprocessing step inside the Pipeline before Logistic Regression.

Scikit-learn Pipeline

- **What:** Chains preprocessing and model steps into one consistent workflow.
- **Why:** Ensures `StandardScaler` is fit only on training data and applied to test data — prevents data leakage. Also makes the workflow clean and reusable.
- **Where:** Combining `StandardScaler` + Logistic Regression into one pipeline object.

Matplotlib + Seaborn

- **What:** Python visualization libraries.
- **Why:** Confusion matrix heatmap makes it easy to see TP/TN/FP/FN visually. Feature coefficient chart shows which features influenced the model most.
- **Where:** Confusion matrix visualization, feature importance chart.

Joblib

- **What:** Python library for saving/loading ML model objects — optimized for large NumPy arrays.
- **Why:** Train once, save, deploy. Next time you need a prediction just load the .pkl file — no retraining needed.
- **Where:** Saving the trained pipeline as phishing_detection_model.pkl.

8. Key ML Concepts Explained Simply

What is Binary Classification?

- Classification = predicting a category. Binary = only two possible categories.
- Our categories: 0 = Legitimate, 1 = Phishing. Model outputs one of these two.
- Different from regression which predicts a number. Here we predict a class.

What does Logistic Regression actually do?

- Takes 30 feature values as input.
- Calculates a probability — for example: 94% probability this is phishing.
- If probability > 0.5 → class 1 (Phishing). If < 0.5 → class 0 (Legitimate).
- The 'Regression' in the name is misleading — it's a classification algorithm.

What is Stratified Split?

- stratify=y ensures both train and test sets have the same proportion of legitimate vs phishing websites.
- Without stratify: by random chance test set might have 90% phishing — skewed evaluation.
- With stratify: proportions are maintained — gives honest evaluation.

What is Data Leakage and how does Pipeline prevent it?

- Data leakage: when information from test data 'leaks' into training — makes model look better than it is.
- If you fit StandardScaler on the full dataset before splitting, scaler has seen test data — that's leakage.
- Pipeline ensures: scaler.fit() only on training data, scaler.transform() on test data. No leakage.

Confusion Matrix — All 4 Cases

- **True Positive (TP):** Website is phishing → Model says phishing. Correct. Good.
- **True Negative (TN):** Website is legitimate → Model says legitimate. Correct. Good.
- **False Positive (FP):** Website is legitimate → Model says phishing. Wrong. User blocked from safe site — annoying but not dangerous.
- **False Negative (FN):** Website is phishing → Model says legitimate. Wrong. User exposed to phishing site — DANGEROUS. This is the worst case for a security system.
- This is why Recall is especially important here — we want to minimize FN (missed phishing sites).

Precision vs Recall — Simple

- **Precision:** Of all websites the model called phishing — how many actually were phishing?
- **Recall:** Of all actual phishing websites — how many did the model catch?
- In a security context, high Recall is more critical — missing a phishing site is worse than a false alarm.

Feature Importance in Logistic Regression

- Logistic Regression doesn't have feature_importances_ like Random Forest.

- Instead, we analyze the model's coefficients — higher magnitude = more influence on prediction.
- Positive coefficient: feature pushes prediction towards phishing. Negative: towards legitimate.
- Example: SSLfinal_State might have coefficient -1.8 (pushes towards legitimate) while having_IP_Address might be +1.5 (pushes towards phishing).

9. Biggest Challenge

- **Challenge:** The dataset is in ARFF format which is not as common as CSV — figuring out how to load it properly in Python.
- **Solution:** Used `scipy.io.arff` module which is purpose-built for ARFF files. Loaded it and converted to Pandas DataFrame for all further processing.
- **Learned:** Always check the dataset format first before writing any preprocessing code. Different formats need different loading approaches.

10. Improvements

- Try ensemble models like Random Forest or XGBoost — often outperform Logistic Regression on tabular data.
- Add GridSearchCV for hyperparameter tuning to squeeze more accuracy.
- Build a simple web interface — user enters a URL, features extracted automatically, model predicts in real time.
- Add ROC-AUC curve as an additional evaluation metric.

11. Follow-up Questions

Q: Why Logistic Regression and not Random Forest or XGBoost?

A: Logistic Regression is interpretable — I can look at the coefficients and understand which features are influencing the prediction and in which direction. For a first version, it's also fast to train and works well on scaled tabular data. Random Forest and XGBoost would likely give higher accuracy and are the natural next step for improvement.

Q: What is supervised learning?

A: Supervised learning means we have labeled training data — inputs paired with correct outputs. The model learns the pattern from these labeled examples. In our case, each website has 30 features AND we know whether it's phishing or legitimate. The model learns what combination of features indicates phishing.

Q: What is ARFF format?

A: ARFF stands for Attribute-Relation File Format. It's commonly used in ML and data mining tools like Weka. It's similar to CSV but includes metadata about each attribute — its name and data type. Python's SciPy library has built-in support for loading ARFF files.

Q: Why is Recall more important than Precision here?

A: In a phishing detection system, a False Negative — missing a phishing site — is much more dangerous than a False Positive — flagging a safe site. If we miss a phishing site, the user gets exposed. If we flag a safe site, the user is just inconvenienced. So we optimize for high Recall even if it comes at some cost to Precision.

Q: What does Joblib do?

A: Joblib serializes the trained model pipeline to a .pkl file. Later, you load this file and the model is ready to make predictions without retraining. This is essential for deployment — you train once, save the model, and any backend can load it to classify new websites in real time.

Q: Why StandardScaler?

A: Different features have very different value ranges. URL_Length might be 50-200 while web_traffic could be in thousands. Without scaling, large-scale features dominate the model and small-scale features get ignored even if they're important. StandardScaler brings all features to mean 0 and standard deviation 1 — equal footing for all features.

Q: What is a Pipeline in Scikit-learn?

A: A Pipeline chains multiple steps — preprocessing and model — into one object. You call `pipeline.fit(X_train, y_train)` and it fits the scaler on training data and trains the model. You call `pipeline.predict(X_test)` and it scales the test data properly before predicting. It prevents data leakage and keeps the code clean.

Q: How would you deploy this?

A: Load the saved .pkl model in a Flask or FastAPI backend. User hits an endpoint with website URL. Backend extracts the 30 features from that URL. Passes features to the loaded model. Model returns 0 or 1. Response sent back as 'Legitimate' or 'Phishing'.

Q: What is overfitting?

A: When a model performs very well on training data but poorly on new unseen data — it has memorized the training examples instead of learning the general pattern. We check for this by comparing training accuracy vs test accuracy. If training accuracy is 99% but test is 70%, the model is overfitting.

PROJECT 3 — HireSense: AI Career Assistant

Tech Stack: React.js, Tailwind CSS, Node.js, MongoDB, Gemini API, Web Speech API

1. One-Line Summary

HireSense is an AI-powered job-readiness platform with 5 modules — ATS resume scoring, mock interview with auto mic activation, AI chatbot, profile summary generation, and career guidance — all powered by Gemini API.

2. 30-60 Second Interview Explanation

■ Say this when asked 'Explain your project'

HireSense is an AI career assistant I built for freshers preparing for placements. Instead of going to 5 different websites for resume review, mock interviews, and career guidance — everything is in one platform.

It has 5 core modules. ATS Resume Analyzer — upload your resume, get a score and detailed feedback on strengths and weaknesses. Mock Interview — system gives a question, 30-second reading timer, mic activates automatically, you answer, and AI scores you on communication, clarity, and confidence. There's also an AI chatbot for career questions, profile summary generation, and career guidance.

Gemini API powers all AI features. Frontend is React.js, backend is Node.js, database is MongoDB.

3. Detailed Features

- **ATS Resume Analyzer:** User uploads resume. Backend extracts text, sends to Gemini with structured ATS evaluation prompt. Gemini returns score + 10+ feedback points — strong areas, weak areas, keyword gaps.
- **Mock Interview Module:** System shows question + 30-sec reading timer. Timer hits zero — mic activates automatically via Web Speech API. User speaks for 30 seconds. Audio converted to text, sent to Gemini. Gemini scores on 3 dimensions — communication, clarity, confidence.
- **AI Chatbot:** Gemini-powered chatbot with prompt engineering to stay focused on career topics only.
- **Profile Generation:** User fills basic form — name, skills, experience, education. Gemini generates a professional LinkedIn-style paragraph. Saved to profile in MongoDB.
- **Career Guidance:** Conversational guidance on career paths, skill roadmaps, role preparation.

4. Complete Workflow

User registers / logs in → JWT token issued

↓ [ATS Path]

Upload resume → Backend extracts text → Sent to Gemini with ATS prompt → Score + feedback returned → Saved in MongoDB → Displayed

↓ [Mock Interview Path]

Question displayed → 30-sec reading timer → Mic auto-activates via Web Speech API → User speaks → Converted to text → Sent to Gemini → Score on 3 dimensions returned

↓ [Profile Generation Path]

User fills form → Sent to Gemini → Professional paragraph generated → Saved in MongoDB profile

5. Tech Stack Explained

Gemini API

- **What:** Google's LLM API — send a prompt, get intelligent text response.
- **Why:** Powers all 5 AI features without building a model from scratch. Good free tier. Strong at text evaluation and generation.

- **Where:** ATS scoring, interview feedback, chatbot, profile generation, career guidance.

Web Speech API

- **What:** Browser's built-in speech recognition — converts mic audio to text.
- **Why:** No external library needed. Works in browser. Activated programmatically when timer ends.
- **Where:** Mock interview — capturing user's spoken answer.

JWT

- **What:** JSON Web Token — stateless authentication.
- **Why:** User logs in once, gets token. Every request carries token. Backend verifies without database lookup. Fast and scalable.
- **Where:** Login, all protected routes.

MongoDB

- **What:** NoSQL document database.
- **Why:** Flexible schema — each user's interview feedback and ATS results have different structures. No rigid tables needed.
- **Where:** Users, interview scores, ATS history, profile summaries.

Node.js + Express.js

- **What:** Backend server and REST API framework.
- **Why:** Handles all API calls — receives frontend requests, calls Gemini, saves to MongoDB.
- **Where:** All backend endpoints.

6. Key Processes

Mock Interview Mic Flow

- `useEffect` in React watches the timer value.
- When timer hits zero: `SpeechRecognition.start()` called programmatically.
- Browser activates mic — user speaks.
- Web Speech API returns transcript in real time.
- On silence or 30-sec limit — recognition stops — transcript sent to backend → Gemini.

Chatbot Prompt Engineering

- System prompt: 'You are a career counselor. Only answer questions about careers, jobs, resumes, and interviews. If asked anything outside this scope, politely redirect.'
- This is prompt engineering — shaping AI behavior through instructions, not actual model training.
- If asked 'Why not build your own AI?': Building and training an LLM requires massive compute and data. Gemini gives state-of-the-art intelligence instantly. We focus on the product, not reinventing AI.

7. Follow-up Questions

Q: What is ATS?

A: ATS stands for Applicant Tracking System. Companies use ATS software to automatically screen resumes before a human sees them — checking for relevant keywords, formatting, experience match. If your resume doesn't pass ATS, it gets rejected before any recruiter reads it. HireSense simulates this evaluation using Gemini.

Q: How does Gemini evaluate a resume?

A: We extract the resume text and send it to Gemini with a structured prompt: 'Evaluate this resume like an ATS system. Check for keyword presence, formatting clarity, experience descriptions, skills alignment, and action verbs. Return a score out of 100 and list strong points and improvement areas.' Gemini returns structured

feedback.

Q: What is prompt engineering?

A: Writing instructions to the AI very carefully so it gives useful, focused responses. Instead of just saying 'evaluate this resume', we write a detailed prompt specifying exactly what to check and how to format the output. Quality of the AI's response directly depends on the quality of the prompt.

Q: Why Gemini over OpenAI?

A: Gemini has a generous free tier which made it practical for development. Response quality for text evaluation tasks is strong. For a project at this scale, it was the right choice. OpenAI's GPT-4 costs money per request.

Q: How is the interview score calculated?

A: We send the user's answer transcript to Gemini with a prompt asking it to score on 3 dimensions — communication, clarity, and confidence — and return specific feedback for each. Gemini evaluates the language quality, structure, and completeness of the answer.

Q: How would you scale HireSense?

A: Add Redis for caching repeated Gemini responses. Use BullMQ job queue for processing resume analysis so multiple users don't block each other. Deploy on AWS with horizontal scaling.

PROJECT 4 — StrideX: E-Commerce Platform

Tech Stack: React.js, Tailwind CSS, Node.js, Express.js, MongoDB, Mongoose, JWT, Razorpay

1. One-Line Summary

StrideX is a full-stack e-commerce platform for footwear — product catalog, cart, JWT auth, order history, and Razorpay payment integration with end-to-end checkout in under 2 seconds.

2. 30-60 Second Interview Explanation

■ Say this when asked 'Explain your project'

StrideX is a full-stack e-commerce platform I built for footwear. It covers the complete shopping experience — browse products across 10+ categories, add to cart, manage orders, and pay through Razorpay.

Authentication uses JWT — user logs in, gets a token, every subsequent request carries that token. Cart and orders are saved in MongoDB linked to the user's ID so everything persists across sessions.

Checkout uses Razorpay test mode — backend creates an order, frontend opens payment modal, user pays, backend verifies the payment signature cryptographically before confirming the order. I built 8+ REST API endpoints covering auth, products, cart, orders, and payments.

3. Complete Workflow

User registers / logs in → JWT token stored in browser

↓

Browses product catalog → Adds items to cart → Cart saved in MongoDB with user ID

↓

Goes to checkout → Backend creates Razorpay order → Returns order ID to frontend

↓

Razorpay modal opens → User pays with test card → Payment processed

↓

Razorpay returns: payment ID + order ID + signature to frontend

↓

Frontend sends all 3 to backend → Backend verifies signature using HMAC-SHA256

↓

If valid → Order saved in MongoDB → Cart cleared → Success page shown

4. Tech Stack Explained

JWT

- **What:** JSON Web Token — self-contained token proving identity. Contains user data encoded and signed with secret key.
- **Why:** Stateless auth — server doesn't store sessions. Any server can verify the token. Scales easily without shared session storage.
- **Where:** Login, registration, all protected routes — cart, orders, checkout.

Razorpay

- **What:** Indian payment gateway supporting UPI, cards, net banking, wallets.

- **Why:** Popular in India, good documentation, test mode for development, straightforward SDK.
- **Where:** Checkout — creating payment orders and verifying completion.

MongoDB + Mongoose

- **What:** NoSQL database + ODM layer adding schema validation.
- **Why:** Flexible document structure for varied product data. Mongoose adds validation and clean query syntax.
- **Where:** Users, products, carts, orders.

Express.js

- **What:** Node.js web framework for REST APIs.
- **Why:** Clean routing, easy JWT middleware on protected routes.
- **Where:** All 8+ REST endpoints.

5. Key Processes

JWT Authentication Flow

- POST /api/auth/login → backend finds user, verifies password with bcrypt
- `jwt.sign({ userId }, SECRET, { expiresIn: '7d' })` → token sent to frontend
- Every protected request: Authorization: Bearer in header
- Middleware decodes token → extracts userId → allows or blocks request

Razorpay Payment Verification

- Backend creates order: `razorpay.orders.create({ amount, currency: 'INR' })`
- Frontend opens modal with order ID → user pays
- Razorpay returns: `razorpay_payment_id`, `razorpay_order_id`, `razorpay_signature`
- Backend verifies: HMAC-SHA256 hash of `orderId|paymentId` using secret must match signature
- If verified → order confirmed → cart cleared

6. Follow-up Questions

Q: What is JWT and how does it work?

A: JWT stands for JSON Web Token. On login, server encodes user data and signs it with a secret key — creating a token. This token goes to the frontend. Every future request includes this token. Backend just verifies the signature — no database lookup needed. Fast and stateless.

Q: Why JWT over sessions?

A: Sessions require server to store session data — when you scale to multiple servers, they need shared session storage. JWT is stateless — any server can verify the token without shared storage. Better for scalable applications.

Q: How does Razorpay payment verification work?

A: After payment, Razorpay gives three values — payment ID, order ID, and a signature. The signature is an HMAC-SHA256 hash of `orderId|paymentId` using our Razorpay secret key. We recreate this hash on our backend and compare it to the signature Razorpay sent. If they match, the payment is genuine. This prevents anyone from faking a successful payment.

Q: What is bcrypt?

A: bcrypt is a password hashing algorithm. Instead of storing raw passwords, we store a bcrypt hash. When user logs in, we hash the entered password and compare to stored hash. Even if database is breached, actual passwords are not exposed.

Q: What is Mongoose?

A: Mongoose is an ODM — Object Document Mapper — for MongoDB. It lets you define schemas for your collections, adds validation, and provides clean query methods. Without Mongoose you'd write raw MongoDB

queries. Mongoose makes code cleaner and prevents bad data from being saved.

Q: Why MongoDB over SQL for e-commerce?

A: Product data varies — some items have sizes, some have colors, some have both. SQL needs fixed table schema. MongoDB stores each product as a document with only the fields it needs. This flexibility is useful for a product catalog.

Q: What happens if payment succeeds but server crashes before saving order?

A: This is a real distributed systems problem. Solution: Razorpay payment IDs are unique. On recovery, check if that payment ID already has an order. If not, create it. For production, I'd use a message queue — payment confirmation goes into a queue, order creation processed with retry logic.

SECTION 5 — EXPERIENCE PREPARATION

EXPERIENCE 1 — Odyssey Homes (Web Developer | March 2026 – Present)

Built a full luxury real estate portal — frontend and backend. Client provided property photos and videos.

- 50+ property listings with detailed pages.
- 3 core modules: property listing, enquiry handling, dynamic content.
- RESTful APIs for browsing and enquiry submission.
- 95% mobile responsiveness score.
- 40% page load improvement via code splitting, asset compression, efficient rendering.

■ *'At Odyssey Homes I work as a Web Developer. I built a luxury real estate portal from scratch — React.js frontend, Node.js backend, MongoDB database. 50+ listings, 3 core modules — listing, enquiry, dynamic content. I optimized performance significantly — 40% faster load through code splitting and asset compression. 95% mobile responsiveness.'*

Q: What is code splitting?

A: Breaking the JavaScript bundle into smaller chunks that load only when needed. Instead of sending all JS at once, only the code for the current page loads. React supports this with `React.lazy()` and dynamic imports. Result — faster initial page load.

Q: How did you achieve 40% load improvement?

A: Three things — code splitting so only necessary JS loads per page, compressing images before serving, and lazy loading images below the fold so they load only when the user scrolls to them.

Q: What does the enquiry module do?

A: When a visitor is interested in a property, they fill a contact form. `POST /api/enquiry` saves it in MongoDB with property ID, user details, timestamp. Property owner can view all enquiries from admin side.

Q: What is mobile responsiveness?

A: Making sure the website works on all screen sizes — phones, tablets, desktops. Used Tailwind CSS responsive utility classes. Achieved 95% score on mobile responsiveness audit.

EXPERIENCE 2 — AIBI (Full Stack Developer | July 2024 – Sept 2025)

Built 3 products at AIBI across web and mobile platforms.

Project A — AI Multiverse (AI Storytelling Platform)

Web-based AI storytelling platform where users create multi-character scripts and generate audio stories.

- User creates story → adds characters (name, gender) → chooses Manual or AI mode.
- **Manual Mode:** User writes dialogue for each character line by line.
- **AI Mode:** User gives brief → Gemini generates complete formatted script.
- Backend converts script to audio using gTTS → combined into one audio file using fluent-ffmpeg.
- Audio stored on Cloudinary → URL saved in Supabase → user plays/downloads.
- Tech: React.js + Node.js + Gemini API + gTTS + Cloudinary + Supabase.

Project B — Loan Application

Mobile banking app for loan management. Full frontend + backend built. RBI permission pending.

- Features: Apply for loan (Aadhaar + PAN fields), Pay EMI, Refer a friend, Loan history (paid/paying/due/completed).
- My role: Built all backend APIs + admin panel for loan verification workflow.

- Security: 2 layers — JWT authentication + role-based authorization.
- KYC: In production would verify via third-party APIs like Digio or Karza against government records.
- Tech: React Native + Supabase + Node.js + Express.js.

Project C — Coupon Stall App

Mobile app for local vendor discounts — similar to McDonald's offers app for any vendor.

- Vendors register, list deals, upload images via Cloudinary.
- Users browse vendor deals and claim coupons.
- Tech: React Native + Node.js + Supabase + Cloudinary.

AIBI Short Answer

■ 'At AIBI I built 3 products. First was AI Multiverse — AI storytelling platform, users create scripts with multiple characters, generate stories via Gemini, convert to audio using gTTS. Second was a loan application — full frontend and backend for loan apply, EMI payment, loan history, 2 security layers. Third was a vendor coupon app similar to McDonald's offers app for local vendors.'

Q: What is gTTS?

A: gTTS stands for Google Text-to-Speech. Converts text to speech using Google's TTS engine and returns an MP3 file. We pass each character's dialogue and get audio back.

Q: How do you combine audio clips?

A: Used fluent-ffmpeg on the Node.js backend. After getting individual MP3 clips from gTTS for each character's line, fluent-ffmpeg merges them in sequence into one complete story audio file.

Q: What is Supabase?

A: Open-source Firebase alternative built on PostgreSQL. Gives relational database, authentication, and storage with a clean API.

Q: What is role-based authorization?

A: Different users have different permissions. Regular users can apply for loans and view their status. Admins can view all applications and approve or reject. Backend checks the user's role from JWT and allows or blocks accordingly.

Q: How does Aadhaar/PAN verification work in real apps?

A: In production, you call a third-party KYC API like Digio or Karza. You send the Aadhaar or PAN number, they verify against government records and return a match result. In our app, fields are collected and stored but not externally verified yet — RBI permission pending.

EXPERIENCE 3 — Sinjan Solutions (Mobile App Developer | Dec 2023 – May 2024)

Built a QR-code based attendance tracking app — also published as research paper in IJRPR.

- 100+ users tracked live attendance.
- Admin generates unique QR per session → students scan → Firebase validates → attendance marked instantly.
- Reduced verification effort by 40%.
- Fixed 3+ critical auth bugs — duplicate attendance, expired QR codes being accepted.

How QR Attendance Works

- Admin generates QR containing encoded session ID + timestamp.
- Student opens app, scans QR → app decodes session ID → sends to Firebase with student ID.
- Firebase validates: Is session active? Already marked? Within time window?
- If valid → attendance marked in Firestore → confirmation shown.

Q: Why Firebase?

A: Firebase provides real-time database updates. When student marks attendance, admin sees it instantly without refreshing. Also handles mobile authentication cleanly.

Q: What auth bugs did you fix?

A: Students could mark attendance multiple times for the same session, and expired QR codes were still being accepted. Fixed by adding session validation checks and timestamp expiry validation on Firebase side.

Q: What is Firestore?

A: Firebase's NoSQL cloud database. Stores data as documents and collections. Supports real-time listeners — when data changes, connected clients update automatically without polling.

EXPERIENCE 4 — Insys Technologies (Web Developer | June 2023 – July 2023)

Internship building responsive web apps with HTML, CSS, JavaScript, PHP, MySQL.

- AJAX for dynamic content without page reloads.
- MySQL query optimization with indexing — 45% reduction in execution time during testing.

Q: What is AJAX?

A: Asynchronous JavaScript — lets you make HTTP requests from JavaScript without reloading the page. Like Google search suggestions appearing without refresh — that's AJAX.

Q: What is database indexing?

A: An index is like a book's table of contents — database jumps directly to relevant rows instead of scanning every row. Makes queries much faster on large tables.

SECTION 6 — RESUME-BASED QUESTIONS

Education

Q: Your CGPA is 9.16. How do you maintain it alongside work experience?

A: I stay consistent throughout the semester rather than cramming before exams. I understand concepts deeply instead of memorizing. Managing time between college, projects, and work taught me to prioritize well.

Q: You have a diploma before your degree. How does that help?

A: My diploma gave me 3 years of strong technical foundation before the degree. I started coding, doing projects, and working at companies earlier than most B.E. students. By the time I entered VESIT, I already had real work experience.

Technical Skills

Q: Difference between React.js and React Native?

A: React.js is for web — renders HTML in the browser. React Native is for mobile — renders native iOS and Android components. Syntax is similar but components differ. Web: div, span. Mobile: View, Text, TouchableOpacity.

Q: What is Next.js and when do you use it over React?

A: Next.js is a framework on top of React with server-side rendering — pages render on the server before sending to browser. Faster initial load and better SEO. Use Next.js for public websites. Plain React for dashboards and logged-in apps.

Q: SQL vs NoSQL?

A: SQL: structured tables, fixed schema, relational data, strong consistency. Good for banking, ERP. NoSQL like MongoDB: flexible documents, schema-less, horizontally scalable. Good for catalogs, social apps, varied data structures.

Q: What cloud services have you used?

A: Vercel and Netlify for frontend, Render and Railway for backend, Cloudinary for media storage, MongoDB Atlas for cloud database, Supabase for PostgreSQL. Basic AWS knowledge.

Q: You know Java, Python, JavaScript. Which is strongest?

A: JavaScript is my strongest — I use it daily for full-stack development. Python I use for ML and AI projects. Java is my primary language for DSA — I've solved 300+ problems using Java on LeetCode and GFG.

Achievements

Q: You conducted a React.js workshop for 60+ students.

A: 4-day hands-on workshop at VES Polytechnic covering React fundamentals, routing, state management, and deployment. Guided participants through building a live project. Teaching forces you to understand things deeply.

Q: You solved 300+ DSA problems. Which area is strongest?

A: Arrays, strings, and trees are my strongest. Comfortable with dynamic programming and graph problems too. Practice on LeetCode and GFG regularly.

Q: You published a research paper.

A: Co-authored a paper on QR Attend — QR-based attendance system — published in IJRPR. Covers system architecture, QR authentication workflow, Firebase integration, and how it reduces manual effort.

Q: Invictus and Need For Code — tell me.

A: Invictus is VESIT's annual technical competition — I reached the finals solving complex problems under time pressure. Need For Code 4.0 at TSEC — I was in top 12. These push you to think fast under pressure.

